

SIH1707 — Geolocation-Based Attendance Tracking App

Build Plan

Organisation: GAIL (India) Ltd, Ministry of Petroleum and Natural Gas **Category:** Software · Miscellaneous **Constraint:** No hardware/software/licenses/data provided by GAIL — free/open-source only. **Reference video:** <https://youtu.be/bmw8unoxA7U>

1. Problem Restated

Build a mobile app that automates employee attendance across multiple GAIL office locations using geolocation, with these hard requirements:

1. **Auto check-in/out** when entering/leaving a 200m radius of an office.
2. **Every check-in pairs with a check-out**, no matter how many times the employee enters/exits in a day.
3. **Manual check-in/out for offsite work**, with the app suggesting nearby known locations from live lat/long.
4. **Total working hours** computed per employee per day.
5. **Tamper-proof, accurate records** with real-time sync and no data loss.

The requirements are explicit and testable — treat this like a spec, not a brief.

2. What Actually Wins This

Everyone's basic version will work — check-in on entering a radius, check-out on exit. That is table stakes, not a differentiator. The competition is won on three things the spec implies but doesn't spell out:

- **Anti-spoofing** — GAIL cannot deploy an attendance system that's trivially fakeable with a mock-location app. This is our single biggest differentiator.
- **The pairing invariant under messy real-world GPS** — boundary jitter, dead phones, force-kills, and inter-office jumps all break a naive implementation.
- **Reliable background operation** — Android Doze/OEM battery killers and iOS background suspension are the actual reason most geofencing apps fail in the field, not the geofencing logic itself.

We build the happy path fast, then spend the majority of remaining time on these three. The demo should **actively try to spoof the app on stage** and show it getting caught — that's the moment that wins the room.

3. Tech Stack (free/open-source only — GAIL provides nothing)

Layer	Choice	Why
Mobile frontend	React Native (Expo, bare workflow if needed for background geofencing)	Fast to scaffold, cross-platform, large ecosystem
Geofencing	<code>react-native-background-geolocation</code> (free tier) or native <code>Geofencing API</code> (Android) + <code>CLLocationCircularRegion</code> (iOS)	OS-level geofencing, not polling — critical for battery + reliability
Maps / geocoding	OpenStreetMap + Nominatim (reverse geocode) + Overpass API (nearby place suggestions)	GAIL explicitly forbids paid APIs — no Google Maps Platform
Backend	FastAPI (Python) or Node/Express	Team likely has FastAPI familiarity already
Database	PostgreSQL + PostGIS	Native geospatial queries (radius checks, distance calcs) instead of hand-rolled Haversine everywhere
Auth	JWT + refresh tokens	Standard, no external dependency
Local offline storage	SQLite (via WatermelonDB or plain <code>expo-sqlite</code>)	Queue events when offline, sync later
Admin dashboard	React + Leaflet.js (OSM tiles, free)	Live map, anomaly queue, timesheets
Push/sync	WebSocket or simple polling + background sync task	Real-time sync requirement

4. Data Model (core tables)

```
employees
  id, name, employee_code, office_id (home office), role

offices
  id, name, lat, lng, radius_m (default 200)

attendance_events
  id, employee_id, office_id (nullable for offsite),
  event_type (check_in / check_out),
  source (auto_geofence / manual_offsite),
  lat, lng, accuracy_m, device_id,
  is_mock_flag, is_anomaly_flag,
  server_ts, client_ts, synced_at

sessions
  id, employee_id, check_in_event_id, check_out_event_id(nullable while open),
  status (open / closed / force_closed),
  total_minutes

offsite_locations
  id, name, lat, lng, added_by (for reuse/suggestions)
```

`sessions` is the reconciliation layer that enforces the pairing invariant — never trust raw events alone for hours calculation.

5. Core Logic Design

5.1 Geofence state machine (solves the pairing invariant)

Do not fire check-in/check-out directly off raw "inside radius" booleans — GPS jitter near a 200m boundary will generate dozens of false transitions.

- Use **hysteresis**: enter threshold at 200m, exit threshold at 220–250m.
- Require a **dwel time** (e.g. signal must persist inside/outside for 60–90s) before committing a state transition.
- Every open session gets **auto-closed** at end-of-day (or after N hours) if no check-out event arrives — flagged as `force_closed` for HR review, not silently dropped.
- If an employee enters Office B while a session at Office A is still open, auto-close A as `force_closed` and open a new session at B — never allow two concurrent open sessions.

5.2 Anti-spoofing (the differentiator — build this early, not last)

- **Mock location detection:** check Android's `Location.isFromMockProvider()` flag on every event; reject/flag events where true.
- **Plausibility checks:** reject a location if implied speed since the last fix exceeds a sane threshold (e.g. >150 km/h) — catches GPS teleportation.
- **Environmental cross-check:** where available, capture visible WiFi BSSIDs and cellular tower IDs alongside GPS. Spoofing GPS is easy; spoofing a consistent radio environment at the same time is not. Flag mismatches.
- **Device attestation:** use Play Integrity API (Android, free) to detect rooted/emulated devices at login.
- **Accuracy filtering:** discard fixes with `accuracy_m` above a threshold (e.g. >50m) rather than trusting every GPS reading blindly.
- All flagged events land in an **admin anomaly queue** — never silently auto-reject, since legitimate GPS drift happens. Human-in-the-loop review.

5.3 Offsite manual check-in

- On manual check-in request, reverse-geocode current lat/long via Nominatim.
- Query `offsite_locations` for previously-used nearby points (Overpass/PostGIS radius query) and suggest them; allow "add new location" if none match.
- Manual events are timestamped and geotagged identically to auto events, just tagged `source = manual_offsite`, and still pass through the same anomaly checks.

5.4 Working hours calculation

- Sum `total_minutes` across all `closed` and `force_closed` sessions per employee per day.
- Force-closed sessions get flagged in the timesheet UI so HR can see estimated vs. confirmed hours.

5.5 Offline-first sync

- All events write to local SQLite immediately regardless of connectivity.
- Background sync task pushes queued events when network returns; server dedupes on a client-generated event UUID (not autoincrement) to handle retries safely.
- Local queue is the source of truth on-device until server ack.

6. Build Timeline (36-hour hackathon)

Hours 0-4 — Setup & scaffolding

- Repo, backend skeleton (FastAPI + Postgres/PostGIS), mobile app skeleton (Expo), auth (JWT).
- Seed `offices` table with 2-3 mock GAIL locations for demo.

- Assign roles (see §8).

Hours 4-10 — Core geofencing happy path

- Native geofence registration for seeded offices.
- Basic state machine: enter → check-in event, exit → check-out event.
- Events sync to backend, sessions table populated.
- Basic "today's status" screen on mobile.

Hours 10-16 — Pairing invariant + reconciliation

- Hysteresis + dwell logic.
- Force-close-on-conflict and end-of-day auto-close.
- Working hours calculation from sessions.

Hours 16-24 — Anti-spoofing layer (priority differentiator)

- Mock-location flag capture.
- Speed/plausibility check.
- Accuracy filtering.
- Anomaly flag written to `attendance_events` + surfaced in admin queue.
- (Stretch, if time: WiFi/cell cross-check, Play Integrity.)

Hours 24-30 — Offsite manual flow + offline sync

- Nominatim reverse geocode + suggestion list.
- SQLite local queue, background sync, dedupe on server.
- Kill network mid-demo and confirm events still record and later sync.

Hours 30-34 — Admin dashboard

- Leaflet map with live employee positions/status.
- Anomaly review queue (approve/reject flagged events).
- Timesheet view, exportable (CSV).

Hours 34-36 — Demo prep

- Script the spoof-and-catch moment (see §7).
- Rehearse the full flow twice, end to end, on the actual devices you'll demo on.
- One person owns the pitch narrative; don't wing it.

7. Demo Script (this is what wins)

1. **Happy path** (60s): walk into the geofence, show auto check-in fire live; walk out, show check-out; show the session in the dashboard with computed hours.

2. **The pairing stress test** (30s): re-enter/exit rapidly near the boundary on stage — show it does *not* generate spurious events, and explain the hysteresis/dwell logic in one sentence.
 3. **The spoof attempt** (60s — the differentiator): turn on Android mock location, "teleport" the device to the office. Show the event land in the **anomaly queue flagged as mock**, not silently accepted. This single moment does more for the pitch than any feature list.
 4. **Offsite flow** (30s): manual check-in from a random location, show the suggested-location list from Nominatim/Overpass.
 5. **Offline resilience** (30s): airplane mode on, check-in still records locally; airplane mode off, show it sync to the dashboard.
 6. **Close on the constraint they actually care about**: "Built entirely on free and open-source infrastructure, per the problem statement — no Google Maps billing, no paid geocoding, deployable as-is."
-

8. Suggested Team Roles

- **Mobile/geofencing lead** — state machine, background execution, this is the hardest and most failure-prone piece; needs the most experienced person.
 - **Backend lead** — API, PostGIS queries, sync/dedupe logic.
 - **Anti-spoofing lead** — can work semi-independently once event schema is fixed; this is the differentiator, don't leave it to "if we have time."
 - **Frontend/dashboard + demo owner** — admin dashboard, and owns rehearsing and delivering the pitch.
-

9. Risks & Mitigations

Risk	Mitigation
Background geofencing unreliable on demo devices (OEM battery killers)	Test on the <i>actual</i> demo phones early, not emulators. Whitelist the app from battery optimization before presenting.
GPS accuracy indoors/at venue is poor	Widen radius for demo purposes if needed, and be upfront about it — judges understand indoor GPS limitations.
Running out of time on anti-spoofing	It's scoped in 3 tiers (mock-flag → plausibility → radio cross-check); ship tier 1 no matter what, treat 2-3 as stretch.
PostGIS setup friction under time pressure	Have a fallback: plain lat/lng + Haversine formula in application code if PostGIS setup stalls past hour 2.
Team can't explain a component under judge questioning	Everyone does a 10-minute walkthrough of every other member's piece before the final rehearsal — no unowned code.

10. Stretch Goals (only after §6 core is done)

- WiFi/cell tower cross-verification for spoofing defence.
- Leave/holiday calendar integration affecting expected hours.
- Push notifications for missed check-out reminders.
- Analytics: attendance trends, late-arrival patterns, per-office heatmap.